

ENGR-E 533

Deep Learning Systems

Module 01

Shallow Neural Network Basics

Minje Kim

Department of Intelligent Systems Engineering

Email: minje@indiana.edu

Website: <http://minjekim.com>

Research Group: <http://saige.sice.indiana.edu>

Meeting Request: <http://doodle.com/minje>



INDIANA UNIVERSITY

**SCHOOL OF INFORMATICS,
COMPUTING, AND ENGINEERING**

The Last Layer

- Function approximation

- Don't worry, we're going to cover the first layer, too
 - And the layers in the middle
- But, why the last layer first?
 - Everything we do with neural networks is about approximating a function

The output of the function $\in \mathbb{R}^{K \times 1} \rightarrow \mathbf{y} = \mathcal{F}(\mathbf{x}) \leftarrow$ Your observed data sample $\in \mathbb{R}^{D \times 1}$

The mapping function you want to know (but you can never know its exact form)

The predicted output $\rightarrow \hat{\mathbf{y}} = \mathcal{G}(\mathbf{x}; \mathbb{W}) \leftarrow$ Parameters

The estimate of the mapping function

Error function of your choice

- The objective function? $\arg \min_{\mathcal{G}} \mathcal{D}(\mathbf{y} || \hat{\mathbf{y}})$
 - What's wrong with it?

- It's easier to work with a parametric function $\arg \min_{\mathbb{W}} \mathcal{D}(\mathbf{y} || \mathcal{G}(\mathbf{x}; \mathbb{W}))$

- The actual optimization is done to reduce the sum of all error network $\arg \min_{\mathbb{W}} \sum_t \mathcal{D}(\mathbf{Y}_{:,t} || \mathcal{G}(\mathbf{X}_{:,t}; \mathbb{W}))$



The Last Layer

- Function approximation

- Today we focus on the function approximation
 - Forget about neural networks just for now
- What I want to show you is
 - We can solve this potentially nonlinear function approximation problem linearly
 - In a certain condition
 - We'll transform the raw input data into feature space
- What kind of functions?
 - Basically any kind of functions
 - Scalar-to-scalar mapping
 - Vector-to-scalar mapping
 - Vector-to-[bounded scalar] mapping
 - Vector-to-vector mapping
 - Vector-to-[bounded vector] mapping
- Hold on.. what about differentiation, optimization, backpropagation, etc?
 - Forget about them
 - For now I want to give you an intuition that things are about template matching



The Last Layer

- Case study #1: scalar-to-scalar mapping, a linear approximation

- Suppose the mapping function between scalar variables x and y is defined by

$$y = \mathcal{F}(x) = \sin(x)$$

- And we don't know it

- What happens if we try to approximate it linearly?

$$\hat{y}_t = \mathcal{G}(x_t; \mathbb{W}) = a_1 x_t + a_0$$

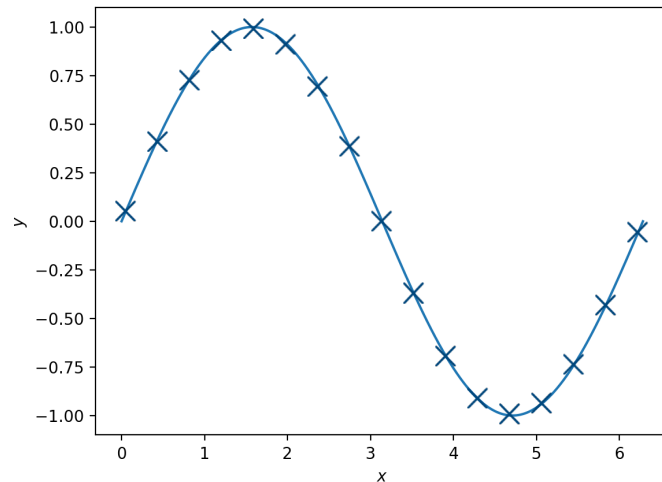
$$\mathbb{W} = \mathbf{a} = [a_1, a_0]^\top$$

$$\hat{y}_t = [a_1, a_0][x_t, 1]^\top$$

- This relationship should hold for all pairs of input and output samples

$$[\hat{y}_0 \ \hat{y}_1 \ \hat{y}_2 \ \cdots \ \hat{y}_{T-1}] = [a_1 \ a_0] \begin{bmatrix} x_0 & x_1 & x_2 & \cdots & x_{T-1} \\ 1 & 1 & 1 & \cdots & 1 \end{bmatrix}$$

$$\Leftrightarrow \hat{\mathbf{y}}^\top = \mathbf{a}^\top \mathbf{X}$$



The Last Layer

- Case study #1: scalar-to-scalar mapping, a linear approximation

$$\hat{\mathbf{y}}^\top = \mathbf{a}^\top \mathbf{X}$$

- What are we going to do with this?

$$\begin{aligned} \arg \min_{\mathbf{W}=\mathbf{a}} \sum_t \mathcal{D}(\hat{y}_t || y_t) &= \arg \min_{\mathbf{a}} \sum_t (\hat{y}_t - y_t)^2 = (\mathbf{y} - \hat{\mathbf{y}})^\top (\mathbf{y} - \hat{\mathbf{y}}) \\ &= \arg \min_{\mathbf{a}} (\mathbf{y}^\top - \mathbf{a}^\top \mathbf{X})(\mathbf{y}^\top - \mathbf{a}^\top \mathbf{X})^\top = \arg \min_{\mathbf{a}} (\mathbf{y}^\top - \mathbf{a}^\top \mathbf{X})(\mathbf{y} - \mathbf{X}^\top \mathbf{a}) \\ &= \arg \min_{\mathbf{a}} (\mathbf{y}^\top \mathbf{y} + \mathbf{a}^\top \mathbf{X} \mathbf{X}^\top \mathbf{a} - 2\mathbf{y}^\top \mathbf{X}^\top \mathbf{a}) \end{aligned}$$

- Then what?

$$\begin{aligned} \frac{\partial \mathbf{y}^\top \mathbf{y} + \mathbf{a}^\top \mathbf{X} \mathbf{X}^\top \mathbf{a} - 2\mathbf{y}^\top \mathbf{X}^\top \mathbf{a}}{\partial \mathbf{a}} &= 2\mathbf{X} \mathbf{X}^\top \mathbf{a} - 2\mathbf{X} \mathbf{y} = 0 \\ \mathbf{a} &= (\mathbf{X} \mathbf{X}^\top)^{-1} \mathbf{X} \mathbf{y} \end{aligned}$$

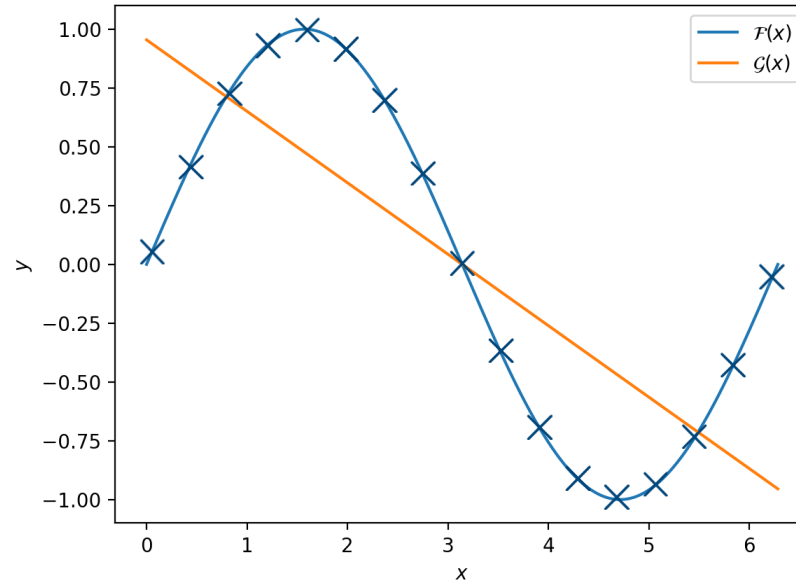
- How can I be sure that this is the solution?
 - The objective function is quadratic
- We've got a closed form solution for this linear model
 - We don't need an iterative optimization algorithm



The Last Layer

- Case study #1: scalar-to-scalar mapping, a linear approximation

○ Life is not that simple..



$$\mathcal{G}(x_t) = -0.30x_t + 0.95$$

The Last Layer

- Case study #2: scalar-to-scalar mapping using nonlinear feature transform

- It's meaningless to model a nonlinear function linearly
- Let's think of a nonlinear transformation of the input
 - So that the mapping between the features and the output can be linear
 - Any idea?
 - There are many different ways..
- I've got a feeling that this function looks like a cubic function
 - Because I was lucky to be able to visualize the data points
 - So, let's try a cubic transformation

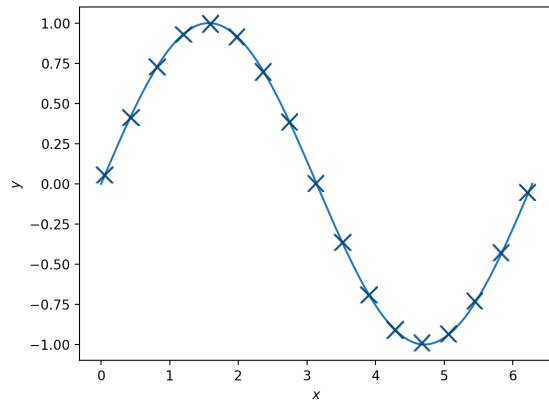
$$[\hat{y}_0 \ \hat{y}_1 \ \hat{y}_2 \ \cdots \ y_{T-1}] = [a_3 \ a_2 \ a_1 \ a_0] \begin{bmatrix} x_0^3 & x_1^3 & x_2^3 & \cdots & x_{T-1}^3 \\ x_0^2 & x_1^2 & x_2^2 & \cdots & x_{T-1}^2 \\ x_0 & x_1 & x_2 & \cdots & x_{T-1} \\ 1 & 1 & 1 & \cdots & 1 \end{bmatrix}$$

$$\Leftrightarrow \hat{\mathbf{y}}^\top = \mathbf{a}^\top \mathbf{X}$$

- How do we solve this?

- Same as before

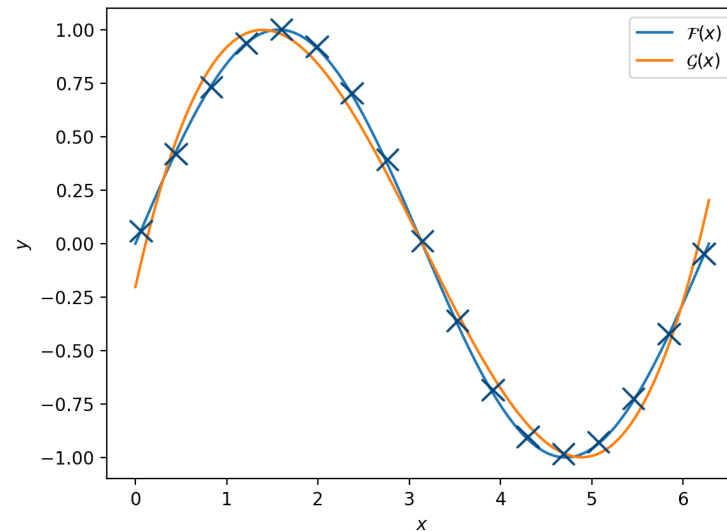
$$\mathbf{a} = (\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{X}\mathbf{y}$$



The Last Layer

- Case study #2: scalar-to-scalar mapping using nonlinear feature transform

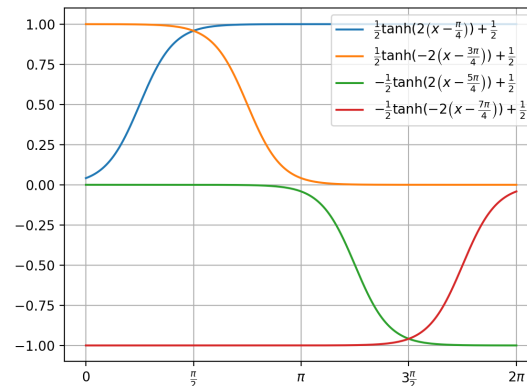
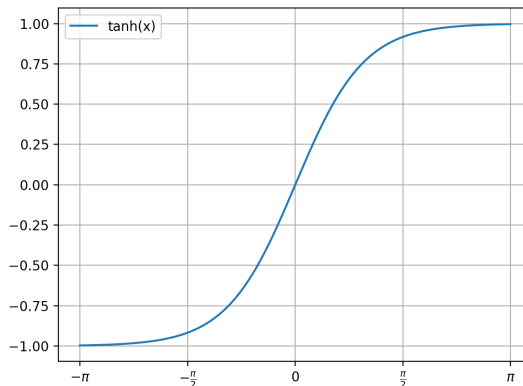
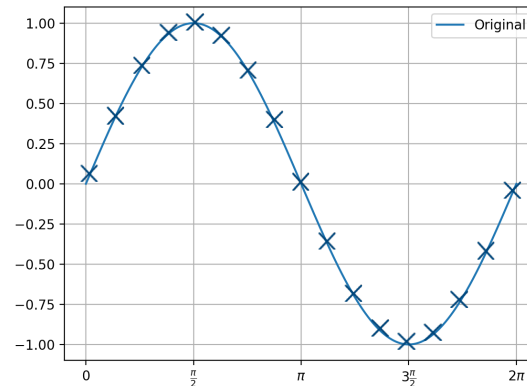
- So far so good..
- Do you see any problem in this approach?
 - You should know the answer if you took MLSP
- You don't always know the mapping function
 - So, I was lucky to know that a cubic function will work
- You should ask me a question at this point
 - "Is there any systematic way that can model *any* function?"
 - But hold on that question for now
- In the procedure we just saw
 - We transformed the 1D data samples into 3D features
 - Then the mapping procedure became a linear function



The Last Layer

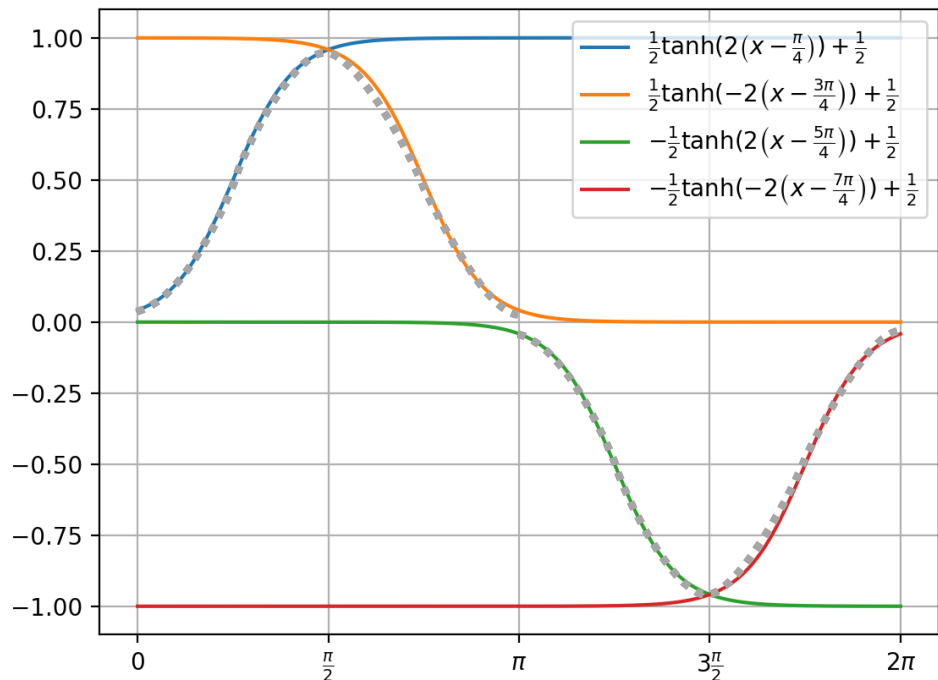
- Case study #3: scalar-to-scalar mapping using *universal* nonlinear feature transform

- Another approach
- This time we're going to use another kind of transform
- Some observations
 - I see a hill and a valley
 - The hill: centered around $\frac{\pi}{2}$
 - The valley: centered around $\frac{3\pi}{2}$
- Let's create them
 - I will use sigmoid functions
 - I rescale and shift them around



The Last Layer

- Case study #3: scalar-to-scalar mapping using *universal* nonlinear feature transform
- If we can somehow combine the “features,” we might recover the original sine function



□ How?



The Last Layer

- Case study #3: scalar-to-scalar mapping using *universal* nonlinear feature transform

- Let's define our features first

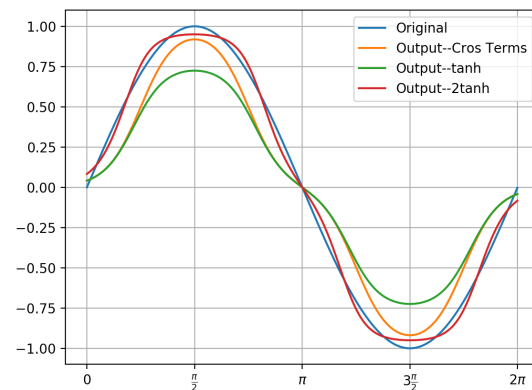
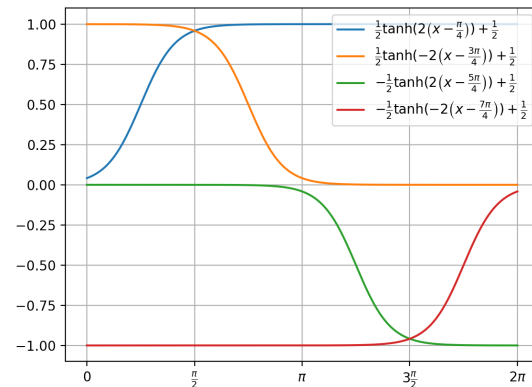
$$h_1 = \frac{1}{2} \tanh\left(2\left(x - \frac{\pi}{4}\right)\right) + \frac{1}{2}$$

$$h_2 = \frac{1}{2} \tanh\left(-2\left(x - \frac{3\pi}{4}\right)\right) + \frac{1}{2}$$

$$h_3 = -\frac{1}{2} \tanh\left(2\left(x - \frac{5\pi}{4}\right)\right) + \frac{1}{2}$$

$$h_4 = -\frac{1}{2} \tanh\left(-2\left(x - \frac{7\pi}{4}\right)\right) + \frac{1}{2}$$

- Can you think of a way to form the sine function?
- $\hat{y} = h_1 h_2 - h_3 h_4$
- $\hat{y} = \tanh(h_1 + h_2 - 1) + \tanh(h_3 + h_4 + 1)$
- $\hat{y} = \tanh(2(h_1 + h_2 - 1)) + \tanh(2(h_3 + h_4 + 1))$
- We can combine a bunch of tanh functions to create valleys and hills!
- How do we find out those coefficients?
 - That's why we need optimization

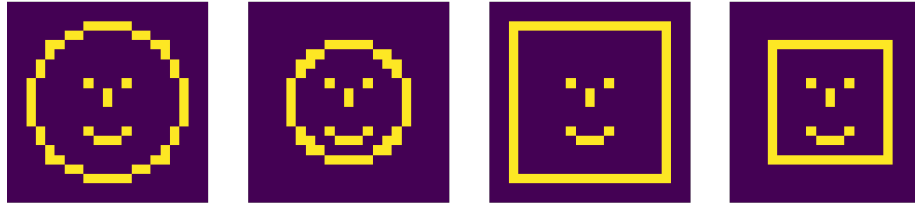


The Last Layer

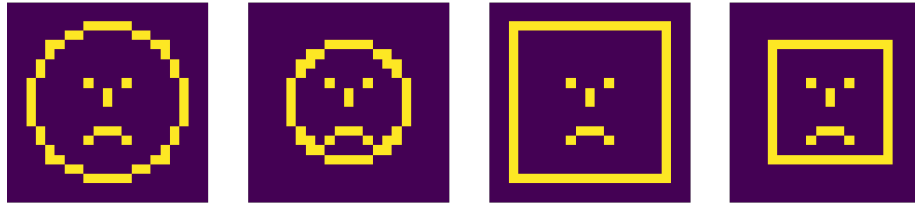
- Case study #4: vector-to-scalar mapping for binary classification

- Roughly speaking, classification can be seen as template matching
 - To see if there's a particular pattern in the example
- Can you figure out what kind of templates are there in the following images?

Class A



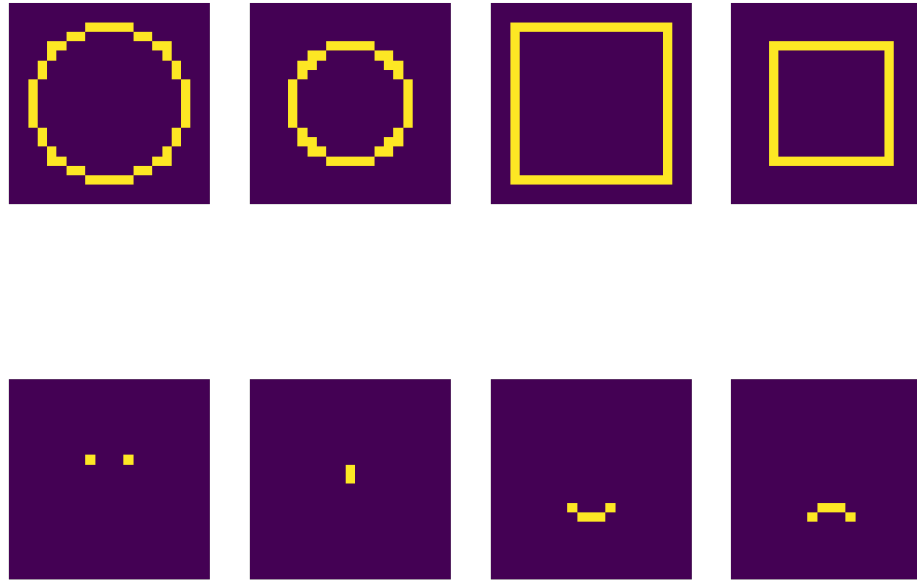
Class B



The Last Layer

- Case study #4: vector-to-scalar mapping for binary classification

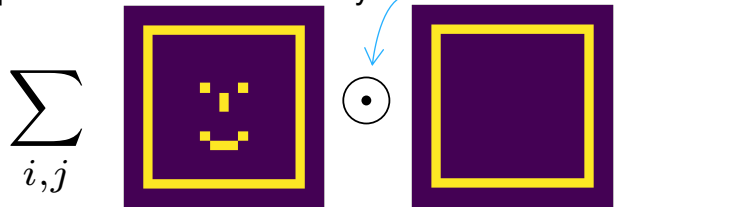
- I feel that there are eight different templates (and features drawn from them)
 - Which one is important for the classification problem?



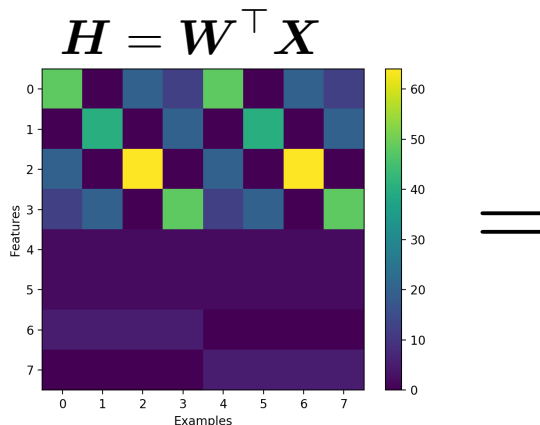
The Last Layer

- Case study #4: vector-to-scalar mapping for binary classification

- Let's check out which one is important
- We do dot product between the template and the image sample
 - Dot product between 2D arrays?



- Vectorize and inner product



=

Each row is a vectorized feature

Column-wise
stacking

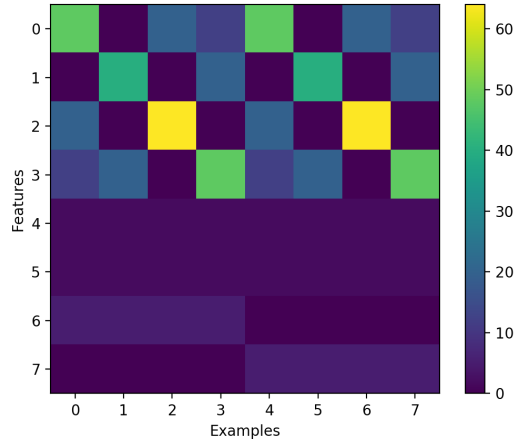
Each column
is a vectorized image



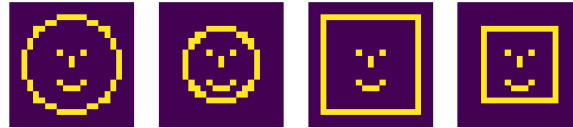
The Last Layer

- Case study #4: vector-to-scalar mapping for binary classification

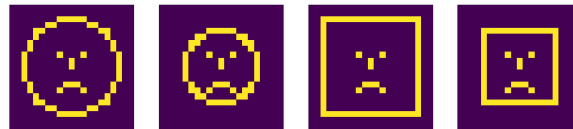
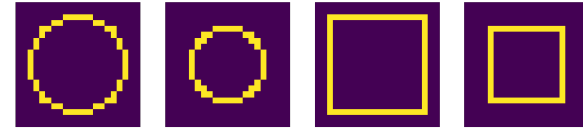
○ What does this mean?



How many features were there originally?



Eight different templates form eight features

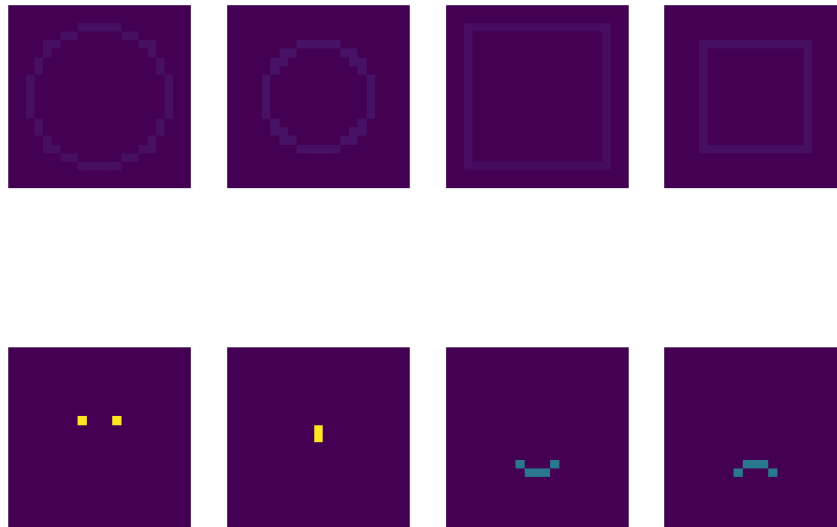


The Last Layer

- Case study #4: vector-to-scalar mapping for binary classification

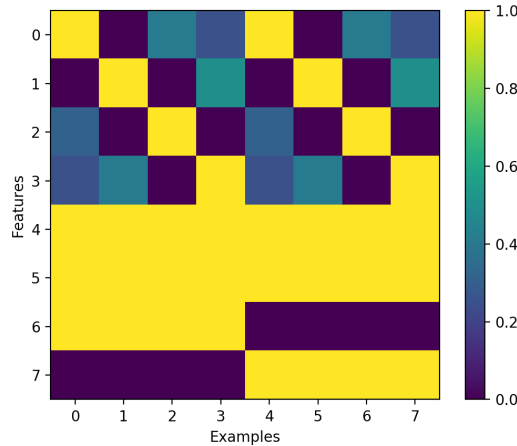
- I finally have a feeling that 6th and 7th features are important
 - But their activations are not strong enough
 - Why? How do we improve it?
- I normalized the templates so that $\tilde{W}_{:,i}$ sums to one

$\tilde{W}$



The Last Layer

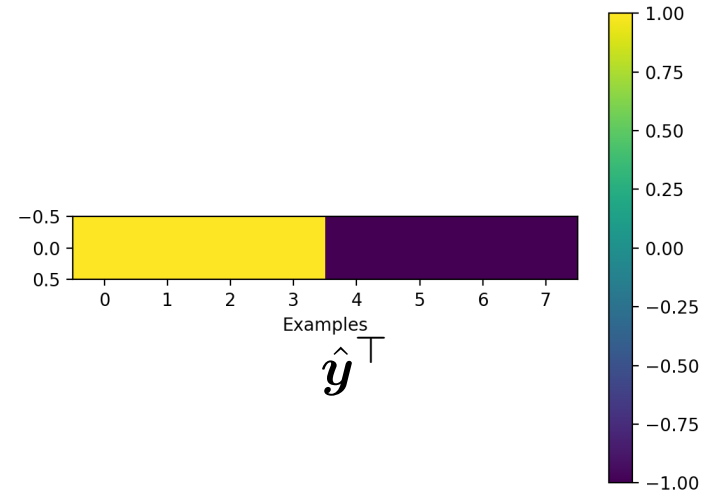
- Case study #4: vector-to-scalar mapping for binary classification



$$H = \tilde{W}^\top X$$

- Then, how do I make a decision?
 - I'll figure out the way to focus only on those important ones
- One way: scalar output $w^{(2)} = [0, 0, 0, 0, 0, 0, 1, -1]^\top$

$$\hat{y}^\top = w^{(2)\top} H$$



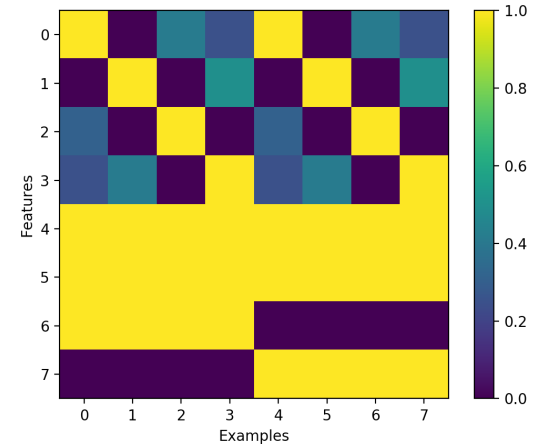
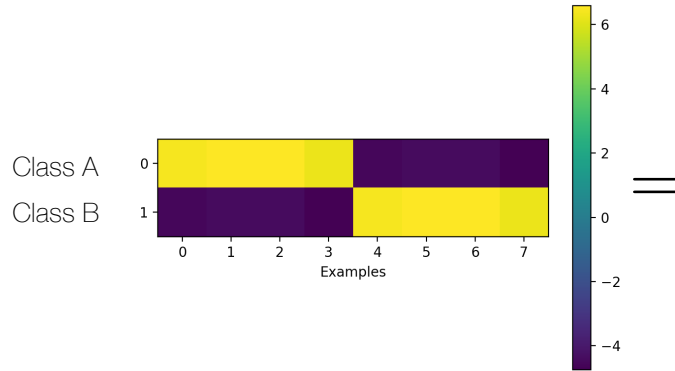
The Last Layer

- Case study #5: vector-to-vector mapping for multiclass classification

- As for the binary classification there's another way to encode the output

$$\mathbf{W}^{(2)} = \begin{bmatrix} -1 & -1 & -1 & -1 & -1 & -1 & 10 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 & -1 & 10 \end{bmatrix}$$

$$\hat{\mathbf{Y}} = \mathbf{W}^{(2)} \mathbf{H}$$



- I want the output to be more intuitive
 - How about some probability?

The Last Layer

- Case study #5: vector-to-vector mapping for multiclass classification

○ Logistic regression

- We want the prediction to be probability vectors

$$P(\text{Class A} | \mathbf{X}_{:,t}) \neq \hat{\mathbf{Y}}_{0,t} = \mathbf{W}_{0,:}^{(2)} \mathbf{H}_{:,t}$$

Why not? Can be negative

$$P(\text{Class B} | \mathbf{X}_{:,t}) \neq \hat{\mathbf{Y}}_{1,t} = \mathbf{W}_{1,:}^{(2)} \mathbf{H}_{:,t}$$

$$P(\text{Class A} | \mathbf{X}_{:,t}) \neq \exp(\hat{\mathbf{Y}}_{0,t}) = \exp(\mathbf{W}_{0,:}^{(2)} \mathbf{H}_{:,t})$$

$$P(\text{Class B} | \mathbf{X}_{:,t}) \neq \exp(\hat{\mathbf{Y}}_{1,t}) = \exp(\mathbf{W}_{1,:}^{(2)} \mathbf{H}_{:,t})$$

Why not? Can be larger than one

$$P(\text{Class A} | \mathbf{X}_{:,t}) = \frac{\exp(\hat{\mathbf{Y}}_{0,t})}{\exp(\hat{\mathbf{Y}}_{0,t}) + \exp(\hat{\mathbf{Y}}_{1,t})}$$

$$P(\text{Class B} | \mathbf{X}_{:,t}) = \frac{\exp(\hat{\mathbf{Y}}_{1,t})}{\exp(\hat{\mathbf{Y}}_{0,t}) + \exp(\hat{\mathbf{Y}}_{1,t})}$$

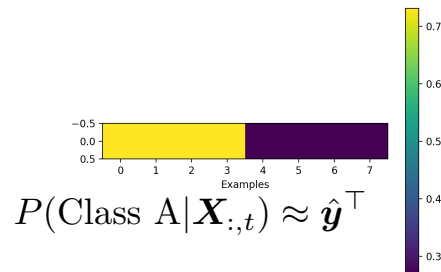
○ $\mathbf{W}^{(2)}$ is overparameterized and we can simplify it

$$\frac{\exp(\hat{\mathbf{Y}}_{0,t})}{\exp(\hat{\mathbf{Y}}_{0,t}) + \exp(\hat{\mathbf{Y}}_{1,t})} = \frac{1}{1 + \exp(\hat{\mathbf{Y}}_{1,t} - \hat{\mathbf{Y}}_{0,t})} = \frac{1}{1 + \exp((\mathbf{W}_{1,:}^{(2)} - \mathbf{W}_{0,:}^{(2)}) \mathbf{H}_{:,t})} = \frac{1}{1 + \exp(\mathbf{w}^{(2)\top} \mathbf{H}_{:,t})} \quad \leftarrow \text{Logistic Function}$$

$$\frac{\exp(\hat{\mathbf{Y}}_{1,t})}{\exp(\hat{\mathbf{Y}}_{0,t}) + \exp(\hat{\mathbf{Y}}_{1,t})} = \frac{\exp(\hat{\mathbf{Y}}_{1,t} - \hat{\mathbf{Y}}_{0,t})}{1 + \exp(\hat{\mathbf{Y}}_{1,t} - \hat{\mathbf{Y}}_{0,t})} = \frac{\exp((\mathbf{W}_{1,:}^{(2)} - \mathbf{W}_{0,:}^{(2)}) \mathbf{H}_{:,t})}{1 + \exp((\mathbf{W}_{1,:}^{(2)} - \mathbf{W}_{0,:}^{(2)}) \mathbf{H}_{:,t})} = \frac{\exp(\mathbf{w}^{(2)\top} \mathbf{H}_{:,t})}{1 + \exp(\mathbf{w}^{(2)\top} \mathbf{H}_{:,t})}$$

$$\mathbf{w}^{(2)} = \mathbf{W}_{1,:}^{(2)} - \mathbf{W}_{0,:}^{(2)}$$

○ This gives us a hint about multiclass classification



The Last Layer

- Case study #5: vector-to-vector mapping for multiclass classification

○ Softmax regression

- Again, we want the output to be a probabilistic vector

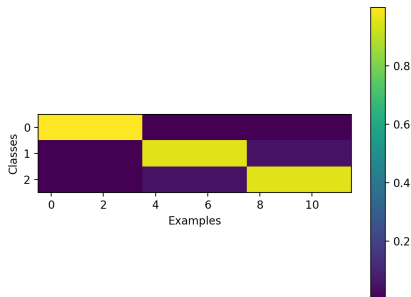
$$\mathbf{Z}_{c,t} = \mathbf{W}_{c,:}^{(2)} \mathbf{H}_{:,t}$$

$$P(\text{Class \#0} | \mathbf{X}_{:,t}) = \frac{\exp(\mathbf{Z}_{0,t})}{\sum_{c=0}^{C-1} \exp(\mathbf{Z}_{c,t})}$$

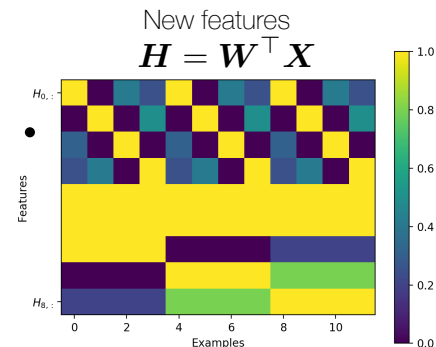
$$P(\text{Class \#1} | \mathbf{X}_{:,t}) = \frac{\exp(\mathbf{Z}_{1,t})}{\sum_{c=0}^{C-1} \exp(\mathbf{Z}_{c,t})}$$

...

$$P(\text{Class \#C-1} | \mathbf{X}_{:,t}) = \frac{\exp(\mathbf{Z}_{C-1,t})}{\sum_{c=0}^{C-1} \exp(\mathbf{Z}_{c,t})}$$



$$= \text{softmax} \left(\begin{matrix} \text{Classes} \\ \begin{matrix} 0 & 1 & 2 \end{matrix} \\ \begin{matrix} 0 & 2 & 4 & 6 & 8 & 10 \end{matrix} \end{matrix} \right) =$$



New dataset

New templates

Class #0



Class #1



Class #2



The Last Layer

- Take-home messages

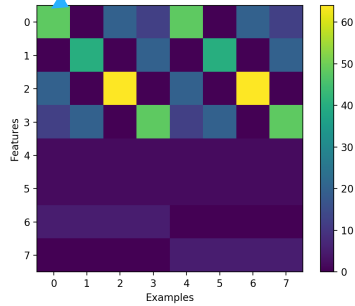
- If you know good features things are easier
 - For regression, you can combine hills and valleys to approximate any funky shape
 - For classification, good features can be created from template matching
 - If you know a good set of templates for the classification
- Some details
 - Sigmoid functions are useful and versatile to create hills and valleys
 - Softmax and logistic functions are useful for the posterior-probability-like output variables
- Potential questions
 - How do we find out those features?
 - Note that I manually found them for this lecture



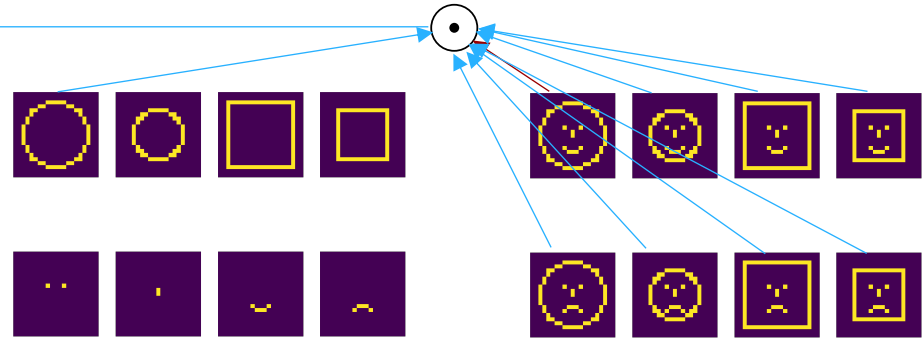
Baby Optimization

- The closed form solution

- Recall the feature extraction process



$$\sum_{i,j}$$



- Or,

$$H = W^T X$$



Baby Optimization

- The closed form solution

- Having $\mathbf{H}$ as the new input we can find another set of weights for the classification job

- Recall that I magically manually found out them

$$\mathbf{w}^{(2)} = [0, 0, 0, 0, 0, 0, 1, -1]^\top \quad \hat{\mathbf{y}}^\top = \mathbf{w}^{(2)\top} \mathbf{H}$$

- If you choose to do optimization we have the pseudo inversed-based solution

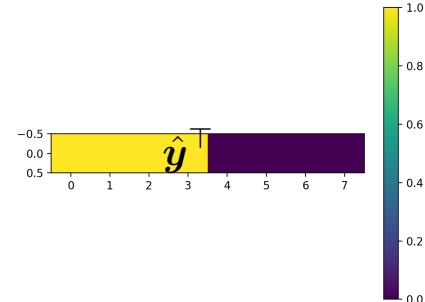
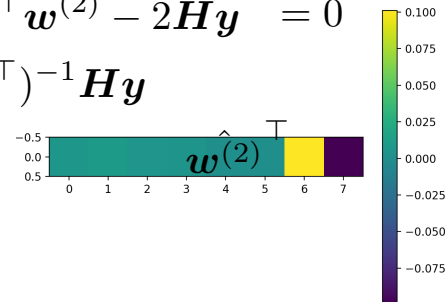
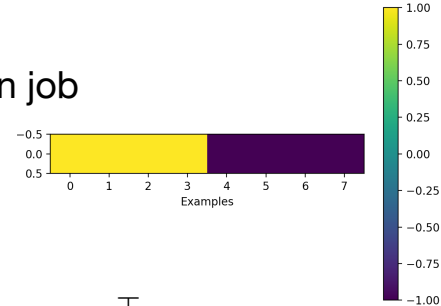
$$\begin{aligned} \arg \min_{\mathbf{w}^{(2)}} \sum_t \mathcal{D}(y_t || \hat{y}_t) &= \arg \min_{\mathbf{w}^{(2)}} \sum_t (y_t - \hat{y}_t)^2 = \arg \min_{\mathbf{w}^{(2)}} (\mathbf{y}^\top - \mathbf{w}^{(2)\top} \mathbf{H})(\mathbf{y}^\top - \mathbf{w}^{(2)\top} \mathbf{H})^\top \\ &= \arg \min_{\mathbf{w}^{(2)}} (\mathbf{y}^\top \mathbf{y} + \mathbf{w}^{(2)\top} \mathbf{H} \mathbf{H}^\top \mathbf{w}^{(2)} - 2\mathbf{y}^\top \mathbf{H}^\top \mathbf{w}^{(2)}) \end{aligned}$$

$$\frac{\partial \mathbf{y}^\top \mathbf{y} + \mathbf{w}^{(2)\top} \mathbf{H} \mathbf{H}^\top \mathbf{w}^{(2)} - 2\mathbf{y}^\top \mathbf{H}^\top \mathbf{w}^{(2)}}{\partial \mathbf{w}^{(2)}} = 2\mathbf{H} \mathbf{H}^\top \mathbf{w}^{(2)} - 2\mathbf{H} \mathbf{y} = 0$$

$$\mathbf{w}^{(2)} = (\mathbf{H} \mathbf{H}^\top)^{-1} \mathbf{H} \mathbf{y}$$

- I define my target output

$$\mathbf{y} = [1, 1, 1, 1, 0, 0, 0, 0]^\top$$



Baby Optimization

- Gradient descent: logistic regression

- Was it too easy?

- Let me make it a little more difficult

- What I'm concerned is the range of $\hat{\mathbf{y}}$, the least mean squared error solution

$$\hat{\mathbf{y}}^\top = \mathbf{w}^{(2)\top} \mathbf{H} \qquad \hat{\mathbf{y}} = ((\mathbf{H}\mathbf{H}^\top)^{-1} \mathbf{H}\mathbf{y})^\top \mathbf{H}$$

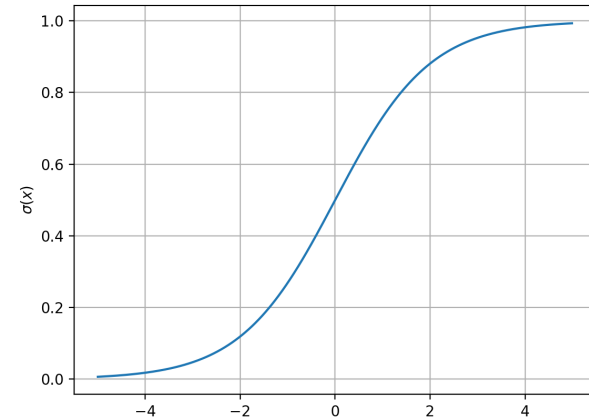
- There's no guarantee that $\hat{\mathbf{y}}$ is between 0 and 1

- Why BETWEEN 0 and 1?

- We care about the probability $P(\text{Class A} | \mathbf{X}_{:,t})$
 - If you're absolutely sure, then this value will be 1 or 0 as in the target variable $\mathbf{y}$
 - In other words, your training samples are with absolutely sure class labels
 - While for your test samples you need to predict the labels by using a posterior probability of the label given the data point

- Any idea?

- Hint: You Already Learned This (YALT)
 - Logistic function!



Baby Optimization

- Gradient descent: logistic regression

- So, we wrap the output values with the logistic sigmoid function

$$\mathbf{z}^\top = \mathbf{w}^{(2)\top} \mathbf{H} \quad \hat{\mathbf{y}}^\top = \sigma(\mathbf{z}^\top)$$

Note that the sigmoid function works element-wise

- Closed form solution?

$$\arg \min_{\mathbf{w}^{(2)}} \sum_t (y_t - \hat{y}_t)^2 = \arg \min_{\mathbf{w}^{(2)}} \left(\mathbf{y}^\top \mathbf{y} + \sigma \left(\mathbf{w}^{(2)\top} \mathbf{H} \right) \sigma \left(\mathbf{H}^\top \mathbf{w}^{(2)} \right) - 2 \mathbf{y}^\top \sigma \left(\mathbf{H}^\top \mathbf{w}^{(2)} \right) \right)$$

- Maybe not a quadratic function anymore
- What does this mean (I mean 'no closed form solution')?
 - There can be many stationary points
 - Impossible to find the global optimum
 - Welcome to the machine learning world!
- Instead, we do numerical optimization
 - We start from randomly initialized parameters
 - Check the error using the current parameters
 - Find the negative gradient direction that reduce the error
 - Update the parameters using the negative gradient direction



Baby Optimization

- Gradient descent: logistic regression

○ $i=0$

1. Initialize parameters with small random numbers
2. Calculate the output using all training samples (i.e. input and target pairs)

$$\mathbf{z}^\top = \mathbf{w}^{(2)\top} \mathbf{H} \quad \hat{\mathbf{y}}^\top = \sigma(\mathbf{z}^\top)$$

3. Calculate the error (cost)

$$\mathcal{E} = \sum_t (y_t - \hat{y}_t)^2$$

4. Update the parameters $\mathbf{w}^{(2)} \leftarrow \mathbf{w}^{(2)} - \rho \frac{\partial \mathcal{E}}{\partial \mathbf{w}^{(2)}}$

Learning rate: how much to move?

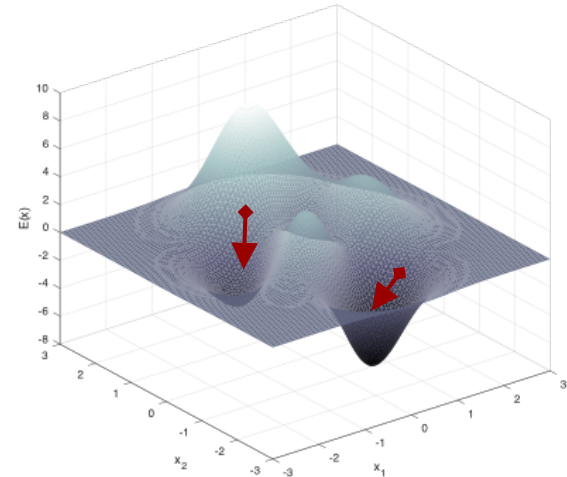
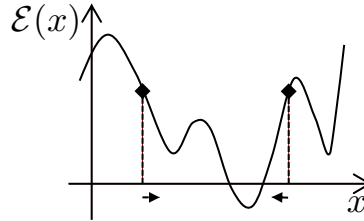
Gradient: in which direction?

○ $i>0$

- Repeat 2-4

○ You should ask two questions

- What is ρ ?
 - Learning rate
- How do we calculate $\frac{\partial \mathcal{E}}{\partial \mathbf{w}^{(2)}}$?
 - It depends on the problem
 - But basically by doing differentiation



Baby Optimization

- Gradient descent: logistic regression

- So, let's do the differentiation

- The chain rule

$$\begin{aligned}\frac{\partial \mathcal{E}}{\partial \mathbf{w}^{(2)}} &= \sum_t \frac{\partial \mathcal{E}}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t} \frac{\partial z_t}{\partial \mathbf{w}^{(2)}} = \sum_t 2(\hat{y}_t - y_t) \sigma'(z_t) \mathbf{H}_{:,t} \\ &= \mathbf{H} \left(\underbrace{(2\hat{\mathbf{y}} - 2\mathbf{y}) \odot \sigma'(\mathbf{z})}_{\text{BP error}} \right)\end{aligned}$$

- The gradient direction for the weights is defined by
 - The inner product of the data matrix and the **backpropagation error**

- Backpropagation error?

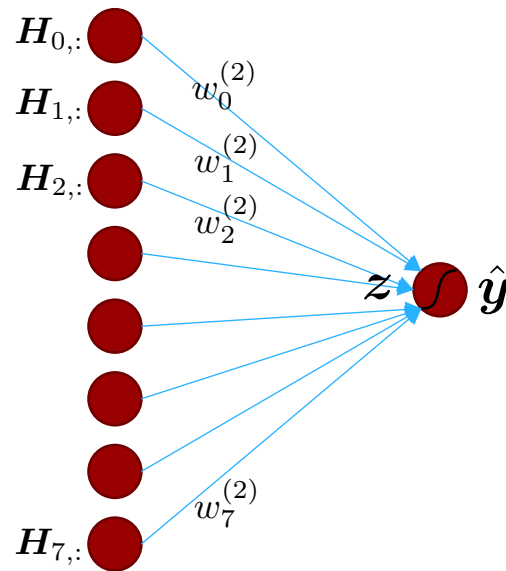
- Partial derivative of the total cost w.r.t. the input to a node

- In this case, $\frac{\partial \mathcal{E}}{\partial z_t}$

$$z_t = \mathbf{w}^{(2)\top} \mathbf{H}_{:,t}$$

$$\hat{y}_t = \sigma(z_t)$$

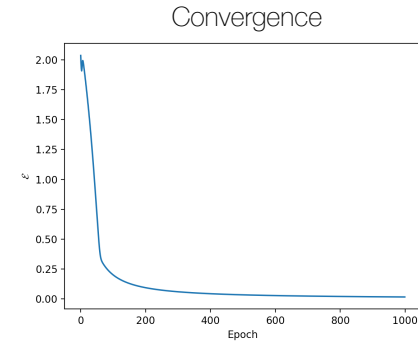
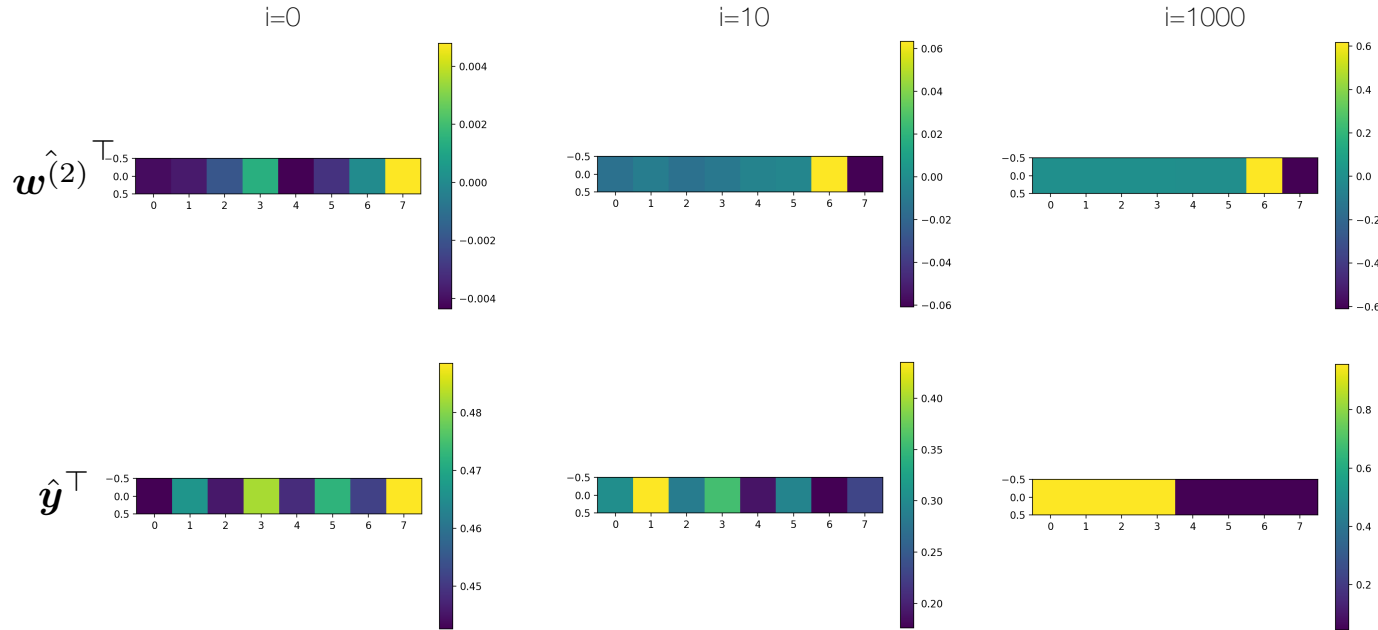
$$\mathcal{E} = \sum_t (y_t - \hat{y}_t)^2$$



Baby Optimization

- Gradient descent: logistic regression

- Parameters and predictions over epochs



Baby Optimization

- Gradient descent: softmax

- Softmax regression is special
 - So is logistic regression
 - The output vector sums to one and nonnegative
 - A probabilistic distribution over classes

- Sum of squared error might not be the best

- Then, why not using a more suitable error metric?

- Cross entropy $-\sum_i q(x_i) \log p(x_i)$

$$\mathcal{E} = -\sum_t \sum_c \mathbf{Y}_{c,t} \log \hat{\mathbf{Y}}_{c,t}$$

- Eventually, we want a partial differentiation of $\mathcal{E}$, but it's not that simple

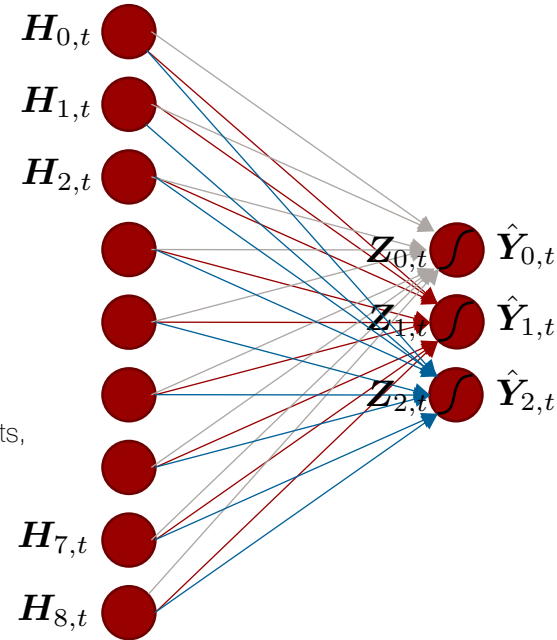
$$\frac{\partial \mathcal{E}}{\partial \mathbf{W}_{j,:}^{(2)}} = \sum_t \sum_c \frac{\partial \mathcal{E}}{\partial \hat{\mathbf{Y}}_{c,t}} \frac{\partial \hat{\mathbf{Y}}_{c,t}}{\partial \mathbf{Z}_{j,t}} \frac{\partial \mathbf{Z}_{j,t}}{\partial \mathbf{W}_{j,:}^{(2)}}$$

$$\frac{\partial \mathcal{E}}{\partial \hat{\mathbf{Y}}_{c,t}} = -\frac{\mathbf{Y}_{c,t}}{\hat{\mathbf{Y}}_{c,t}} \quad \frac{\partial \hat{\mathbf{Y}}_{c,t}}{\partial \mathbf{Z}_{j,t}} = ?$$

Note: j and c are the same index for the three output units, but j is for the input to the units and c is for the output. For example, $\mathbf{W}_{0,:}^{(2)}$ is for the gray arrows

$$\begin{aligned} P(\text{Class } \#0 | \mathbf{X}_{:,t}) &= \frac{\exp(\mathbf{Z}_{0,t})}{\sum_{c=0}^{C-1} \exp(\mathbf{Z}_{c,t})} \\ P(\text{Class } \#1 | \mathbf{X}_{:,t}) &= \frac{\exp(\mathbf{Z}_{1,t})}{\sum_{c=0}^{C-1} \exp(\mathbf{Z}_{c,t})} \\ &\vdots \\ P(\text{Class } \#C-1 | \mathbf{X}_{:,t}) &= \frac{\exp(\mathbf{Z}_{C-1,t})}{\sum_{c=0}^{C-1} \exp(\mathbf{Z}_{c,t})} \end{aligned}$$

Summation over c makes differentiation difficult



Baby Optimization

- Gradient descent: softmax

- Any $\hat{\mathbf{Y}}_{c,t}$ involves all $\mathbf{Z}_{j,t} \forall j$

$$\frac{\partial \hat{\mathbf{Y}}_{c,t}}{\partial \mathbf{Z}_{j,t}} = \frac{\exp(\mathbf{Z}_{c,t}) (\sum_i \exp(\mathbf{Z}_{i,t})) - (\exp(\mathbf{Z}_{c,t}))^2}{(\sum_i \exp(\mathbf{Z}_{i,t}))^2}$$

$$= \hat{\mathbf{Y}}_{c,t}(1 - \hat{\mathbf{Y}}_{c,t}) = \hat{\mathbf{Y}}_{j,t}(1 - \hat{\mathbf{Y}}_{j,t})$$

$$\frac{\partial \hat{\mathbf{Y}}_{c,t}}{\partial \mathbf{Z}_{j,t}} = -\frac{\exp(\mathbf{Z}_{c,t}) \exp(\mathbf{Z}_{j,t})}{(\sum_i \exp(\mathbf{Z}_{i,t}))^2} \quad \text{if } j \neq c$$

$$= -\hat{\mathbf{Y}}_{c,t} \hat{\mathbf{Y}}_{j,t}$$

$$\frac{\partial \mathcal{E}}{\partial \mathbf{W}_{j,:}^{(2)}} = \sum_t \sum_c \frac{\partial \mathcal{E}}{\partial \hat{\mathbf{Y}}_{c,t}} \frac{\partial \hat{\mathbf{Y}}_{c,t}}{\partial \mathbf{Z}_{j,t}} \frac{\partial \mathbf{Z}_{j,t}}{\partial \mathbf{W}_{j,:}^{(2)}}$$

$$= \sum_t \left(-\frac{\mathbf{Y}_{j,t}}{\hat{\mathbf{Y}}_{j,t}} \hat{\mathbf{Y}}_{j,t}(1 - \hat{\mathbf{Y}}_{j,t}) + \sum_{c \neq j} \frac{\mathbf{Y}_{c,t}}{\hat{\mathbf{Y}}_{c,t}} \hat{\mathbf{Y}}_{c,t} \hat{\mathbf{Y}}_{j,t} \right) \mathbf{H}_{:,t}^\top$$

$$= \sum_t \left(-\mathbf{Y}_{j,t}(1 - \hat{\mathbf{Y}}_{j,t}) + \sum_{c \neq j} \mathbf{Y}_{c,t} \hat{\mathbf{Y}}_{j,t} \right) \mathbf{H}_{:,t}^\top$$

$$= \sum_t \left(-\mathbf{Y}_{j,t} + \sum_c \mathbf{Y}_{c,t} \hat{\mathbf{Y}}_{j,t} \right) \mathbf{H}_{:,t}^\top = (\hat{\mathbf{Y}}_{j,:} - \mathbf{Y}_{j,:}) \mathbf{H}^\top$$

$$\frac{\partial \mathcal{E}}{\partial \mathbf{W}^{(2)}} = (\hat{\mathbf{Y}} - \mathbf{Y}) \mathbf{H}^\top$$

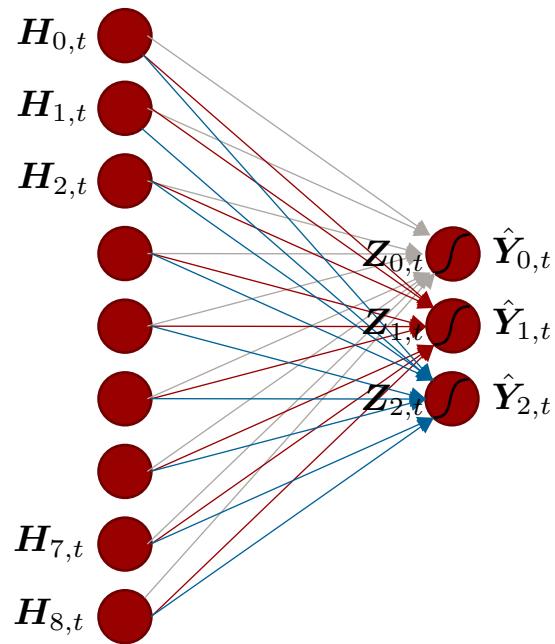
$$P(\text{Class } \#0 | \mathbf{X}_{:,t}) = \hat{\mathbf{Y}}_{0,t} = \frac{\exp(\mathbf{Z}_{0,t})}{\sum_{c=0}^{C-1} \exp(\mathbf{Z}_{c,t})}$$

$$P(\text{Class } \#1 | \mathbf{X}_{:,t}) = \hat{\mathbf{Y}}_{1,t} = \frac{\exp(\mathbf{Z}_{1,t})}{\sum_{c=0}^{C-1} \exp(\mathbf{Z}_{c,t})}$$

...

$$P(\text{Class } \#C-1 | \mathbf{X}_{:,t}) = \hat{\mathbf{Y}}_{C-1,t} = \frac{\exp(\mathbf{Z}_{C-1,t})}{\sum_{c=0}^{C-1} \exp(\mathbf{Z}_{c,t})}$$

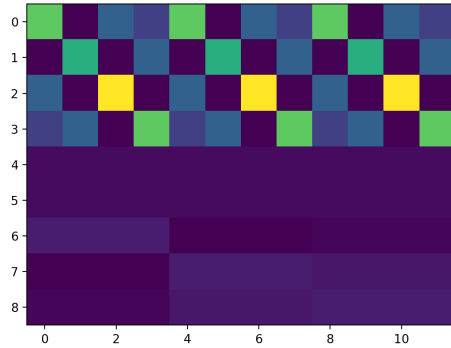
if $j = c$



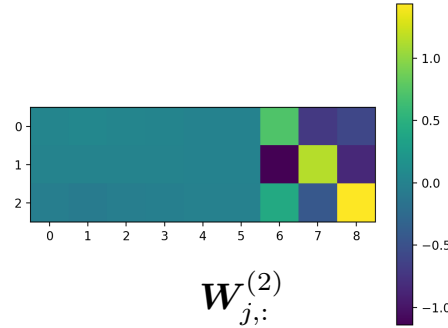
Baby Optimization

- Gradient descent: softmax

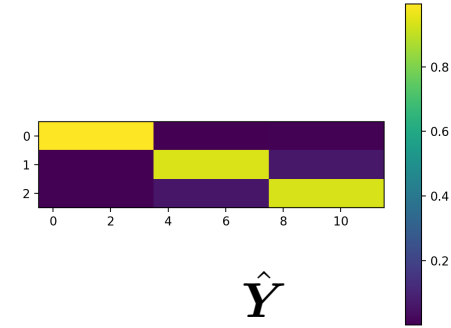
○ Parameter estimates: the three-class case



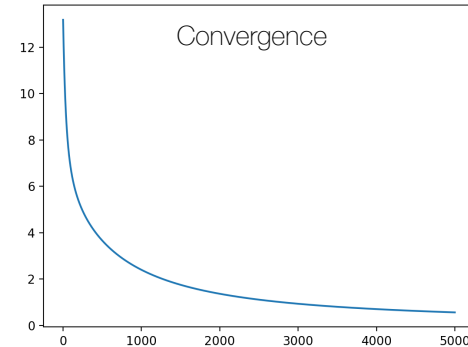
$$H = W^T X$$



$$W_{j,:}^{(2)}$$



$$\hat{Y}$$



Take-home Messages

- Parameter estimation for nonlinear models involve complicated optimization procedure
 - No global optimum
 - Differentiation
 - Learning rate
 - Gradient directions
 - Initialization
 - Computational cost
- In the last layer the choice of the activation function and cost function matters
 - You want to transform the output variable so that
 - Its dynamic range matches the target variable
 - The cost function needs to be chosen accordingly



Deep Learning Frameworks

- GPU computing

- A little bit about GPU computing
 - GPUs are different from CPUs in terms of their number of cores
 - K80 has 2496 cores
 - A better choice for a task with many simple sub-tasks, such as matrix multiplication
 - Each core is much less powerful than a usual CPU core
- Common packages you might need
 - TensorFlow or PyTorch
 - You may want to use some even higher level wrappers like Keras
 - Jupyter
 - A nice interactive development framework
 - Google Colab comes with something similar to this



Deep Learning Frameworks

- Baby TensorFlow

- **Computational graphs**
 - That consist of tensors and operations on them
- Building and running the computational graphs are separate
 - You define the graph first
 - This procedure is to represent what you want to do symbolically
 - It doesn't do any computation
 - Running the graph
 - Visit the graph nodes as required by the session's **run** method
 - Finally does the computation
- What for?
 - One obvious reason is to make your life easier by doing differentiation for you
 - So, it does the differentiation on the symbolic representations
 - And then actually calculate the gradient when you run the graph
- TF 2.x “eager” mode
 - TF 2.x comes with the “eager” mode, where one can skip the graph construction



Deep Learning Frameworks

- Baby TensorFlow

○ So what is a **Tensor**?

- A datatype that TF uses, something pretty similar to `np.array`
 - For example, `tf.constant` returns a tensor that doesn't change its values once initialized
- `tf.Variable`: takes a tensor and then turns into something more usable
 - With the `trainable` flag on, now the tensor can be update via a learning algorithm over iterations e.g., NN weights

○ For example,

- You want to train a classifier from a very large training dataset

$$Y \approx \mathcal{G}(X; W)$$

This will be a very large data matrix

$$\frac{\partial \mathcal{E}}{\partial W^{(2)}} = (\hat{Y} - Y)H^T$$

A heavy matrix operation, maybe too large for the main memory

- So, you want to do something called **Stochastic** Gradient Descent (SGD), where
 - You sample a data point and calculate the gradient by using it
 - Or, you divide the entire data matrix into smaller pieces, called **minibatches** and calculate gradients for each of them
- Then,
 - `tf.Variable`: $W^{(2)}$ ← You can set TF to update them during the run (of course you can initialize them as you want)
 - `tf.constant`: $H_{:,t}, Y_{:,t}$ ← You need to feed this SGD samples or minibatches at every iteration

○ In practice you may just use Keras wrappers and forget about this detail



Deep Learning Frameworks

- Baby TensorFlow

○ In TF 1.x

- There used to be a concept called **session** where all the variables and operations are actually taken care of
- `tf.session.run(fetches, feed_dict=None, ...)`
 - `fetches`: The graph element you want to run, e.g. loss
 - `feed_dict`: You can assign some values to the placeholder tensors (e.g., input)

○ Chain reaction

- Running a graph element means running all the operations and evaluating tensors that are necessary for the fetched one

○ For example,

- You are interested in the training cost $\mathcal{E} = - \sum_t \sum_c Y_{c,t} \log \hat{Y}_{c,t}$
- You define this cost as a graph element
- If you do `sess.run()` on it, you need to know $\hat{Y}$, the prediction of the class labels
- `sess.run()` traverses the graph to get this done

○ `tf.gradients(cost, variables)`

- Does the differentiation w.r.t. the variables
- Most of the time you'll use a wrapper that calls this function internally



Deep Learning Frameworks

- Baby TensorFlow

- In TF 2.x

- The concept of `session` is deprecated, so is graph construction and execution (mostly)

- Eager execution

- TF executes the code line by line, much more like in Python (actually PyTorch)

- For example,

- `x = tf.Variable(3.0)` ← By default, this is trainable

- `with tf.GradientTape() as tape:` ← TF "automatically" computes the gradients for the trainable things in this block

- `y = x**2`

- `dy_dx = tape.gradient(y, x)` ← Actually, computes gradients

- `dy_dx.numpy()`

- What's special about the `with tf.GradientTape() as tape:` is that

- TF 2.x still computes the value of `y` as the eager execution mode (TF 1.x didn't)
 - But, "taping" the autograd-related operations is not done



Deep Learning Frameworks

- Baby PyTorch

- Difference between TF and PyTorch
- The look
 - TF 1.x: you construct the graph first and then run the graph
 - PT: it does construct the graph but things are more seamless
 - Therefore, the source code is more similar to the regular numpy codes
 - TF 2.x has evolved into this direction too
- The tensors
 - PT: a tensor has three specifiers
 - `torch.dtype`: integers, floating-points, etc.
 - `torch.device`: GPU or CPU
 - `torch.layout`: dense or sparse
- GPU computing
 - TF: if the same operation has two kernels for both CPU and GPU computing, GPU version gets the priority
 - PT: different ways are there but basically you define it by using `torch.device` the keyword
 - e.g. `y = torch.tensor([1., 2.]).cuda()`, `a = torch.tensor([1., 2.], device=torch.device('cuda:0'))`, `y = torch.tensor([1., 2.]).to(device=torch.device('cuda:0'))`



Deep Learning Frameworks

- Baby PyTorch

○ The gradients

- TF: `tf.GradientTape`
- PT: all tensors have their own `.grad` component that holds the actual gradient values
 - It's called "autograd" so things are still automatic, but treated differently
 - When `.backward()` method is called, the node is differentiated w.r.t. the leaf nodes in a recursive way
 - To do so, the forward pass records the input to the node
 - The backward pass calculate the gradients of all the intermediate nodes on the way back
 - ONLY IF THE TENSOR'S `requires_grad` FLAG IS ON
 - `x = torch.tensor([[1., -1.], [1., 1.]], requires_grad=True)`
 - `x = torch.tensor([[1., -1.], [1., 1.]])`
`x.requires_grad_()`



Deep Learning Frameworks

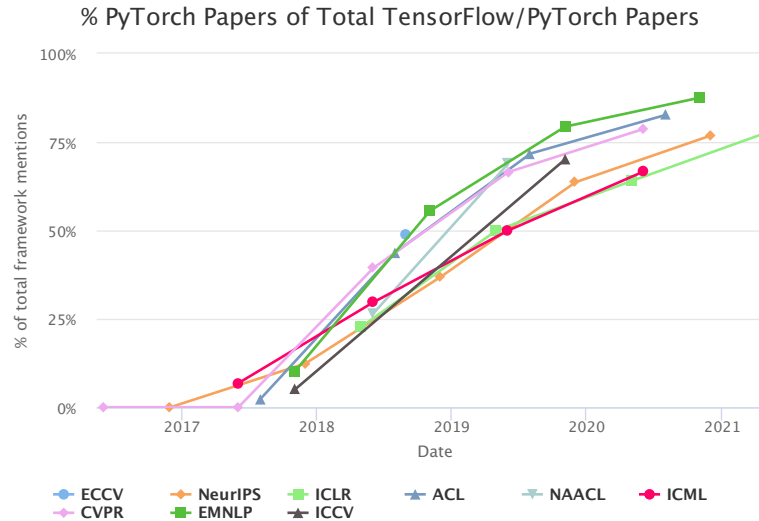
- TF versus PT

○ TensorFlow

- Google
- Seems to have a larger user community
- Said to be better in building a serious project

○ PyTorch

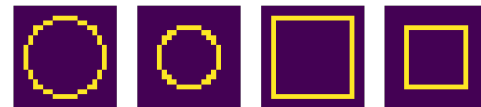
- Facebook and the other supporting organizations
- Rapidly evolving, especially in the academic communities
- Easier to quickly see the proof of concept



The First Layer

- Unsupervised feature learning

- Before deep learning, a common practice was to extract features and then learn a supervised model
 - Source separation
 - Convert the time domain audio signals into matrices by using Short-Time Fourier Transform (STFT) and then learn dictionaries
 - Object recognition
 - Extract a bunch of different features (e.g. HoG, SIFT, etc) and then build a classifier
 - Sentiment analysis
 - Preprocess the text, learn topics, and then build a classifier
 - Speech recognition
 - Extract Mel-Frequency Cepstrum Coefficients (MFCC) and then learn Hidden Markov Models
- YALT
 - In our baby facial expression recognition problem we manually extracted the features first
 - And then built a softmax classifier



The First Layer

- Unsupervised feature learning

- Today we will learn how to systematically learn those features from raw data
 - We call this procedure **unsupervised feature learning**
 - Don't worry, we're not getting away from neural networks
- Why unsupervised?
 - Supervision here means human intervention to solve the problem
 - e.g. When you train a classifier, you need a bunch of pairs: a data sample (facial image) and its label (happy or sad)
 - You let the model know how you think of the problem
 - Mathematically, this can be done by either learning the mapping function between the data sample and its label
$$\mathbf{y} = \mathcal{F}(\mathbf{x}) \quad \hat{\mathbf{y}} = \mathcal{G}(\mathbf{x} ; \mathbb{W})$$
 - Or, finding the parameters that maximize the *a posteriori* probability
$$P(\mathbf{y}|\mathbf{x}; \Theta)$$
 - Unsupervised learning
 - You don't have the human intervention, or the labels
 - Then, what are we learning?
 - A model that best describes the data

The First Layer

- Unsupervised feature learning

○ Modeling the data

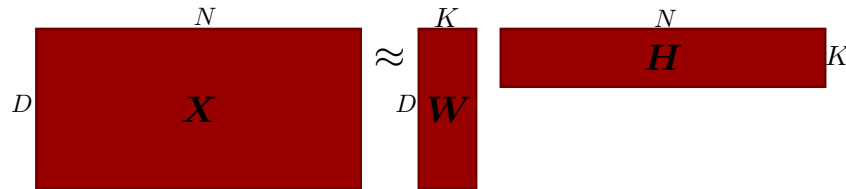
- If we use a probabilistic distribution it's to find out the maximum likelihood solution

$$P(\mathbf{x}; \Theta)$$

- Often, this can be viewed as a latent variable analysis with K latent variables, too

The data matrix $\rightarrow \mathbf{X} \approx \mathbf{W}\mathbf{H} \leftarrow$ The activation of the latent variable

The representative vectors of the latent variable



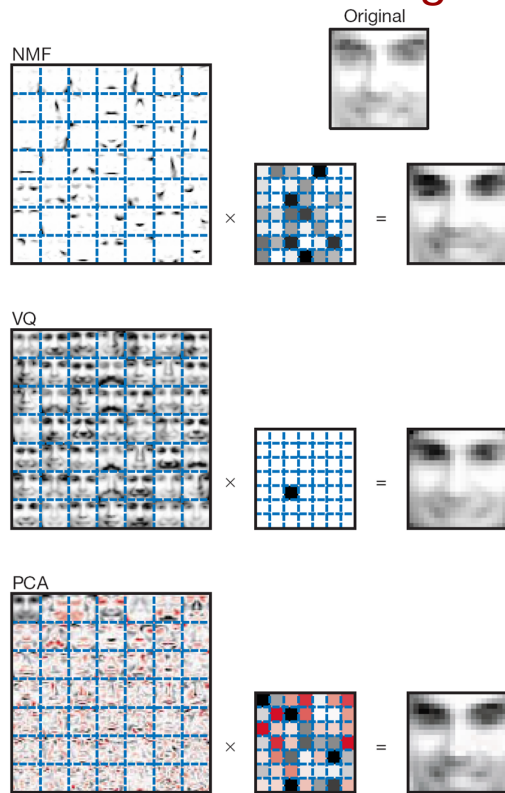
○ For example

- If you find the eigenvectors of $\mathbf{X}\mathbf{X}^T$, that will correspond to $\mathbf{W}$
 - $\mathbf{W}^T\mathbf{W} = \mathbf{I}$ and, therefore, $\mathbf{W}^T\mathbf{X} = \mathbf{H}$. Also, rows of $\mathbf{H}$ are ordered in their variances
 - Principal Component Analysis
 - $\mathbf{H}$ is a set of lower dimensional features that can still describe the original distribution
- If $\mathbf{X}_{:,n} \sim \sum_k \mathbf{H}_{k,n} \mathcal{N}(\mathbf{W}_{:,k}, \Sigma_{:,k})$ and $\mathbf{H}_{:,n}$ is a one hot vector
 - Gaussian Mixture Model (a.k.a. Vector Quantization)
- If you find $\mathbf{W}$ and $\mathbf{H}$ that minimize the approximation error, but are nonnegative at the same time
 - Nonnegative Matrix Factorization
 - $\mathbf{H}$ gives you the parts-based representation



The First Layer

- Unsupervised feature learning



NMF estimates parts-based representations, something like Lego blocks.
Reconstruction is a linear combination of them, but subtraction is not allowed.

VQ finds a bunch of cluster means.
Reconstruction is choosing the most similar mean.

PCA finds the holistic eigenfaces.
From the important one down to the subtle ones.

The First Layer

- Unsupervised feature learning

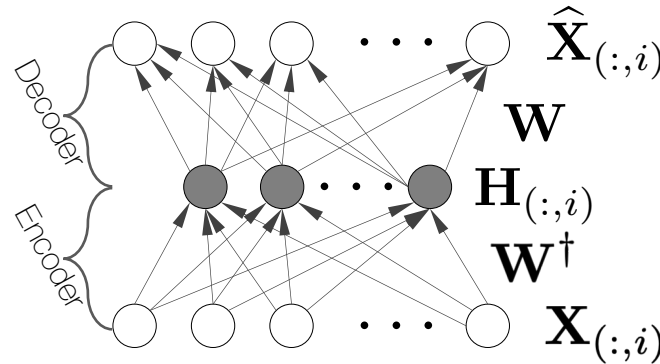
- There are so many different ways to solve the linear approximation $X \approx WH$
- So, eventually it's all about how to constrain the problem

$$\arg \min_{W, H} \mathcal{D}(X || WH) + \lambda_W f(W) + \lambda_H g(H)$$

- For clustering, you want a super sparse column vectors in H so that the data sample is one of the cluster means, deviated accordingly
- For PCA, you want the after-projection-samples are with maximal variances
- For NMF, you constrain W, H to be nonnegative (element-wise)
- ...
- It is convenient to assume that there exists some kind of pseudo inverse of W
 - For PCA, $W^\dagger = W^\top$
 - For NMF, $W^\dagger = W^\top$ depending on the choice of the error function $H \leftarrow H \odot \frac{W^\top X}{W^\top W H}$
- If you assume this pseudo inverse of the basis vectors, you can think of the projection as a way to convert your data into features $W^\dagger X \approx W^\dagger W H \approx H$

The First Layer

- Autoencoder: neural network-based unsupervised learning



$$\arg \min_{\mathbf{W}, \mathbf{W}^\dagger, \mathbf{H}} \mathcal{D}(\mathbf{X} || \mathbf{W} \mathbf{W}^\dagger \mathbf{X}) + \lambda_{\mathbf{W}} f(\mathbf{W}) + \lambda_{\mathbf{W}^\dagger} f(\mathbf{W}^\dagger) + \lambda_{\mathbf{H}} g(\mathbf{H})$$

- Once the objective function is setup in this way, you can estimate the parameters using SGD
 - TF or PT will take care of this part using automatic gradient calculation

The First Layer

- Autoencoder: neural network-based unsupervised learning

- Demo

- Sparse coding
 - NMF



The First Layer

- Nonlinearity in neural networks

- So far I haven't said anything about nonlinearity in the lower layers (at least officially)
- Why do we need it?
 - Because stacked linear models form yet another linear model

- Stacked linear models

$$x_1^{(2)} = w_{11}^{(1)} x_1^{(1)} + w_{12}^{(1)} x_2^{(1)} + b_1^{(1)}$$

$$x_2^{(2)} = w_{21}^{(1)} x_1^{(1)} + w_{22}^{(1)} x_2^{(1)} + b_2^{(1)}$$

$$y = w_1^{(2)} x_1^{(2)} + w_2^{(2)} x_2^{(2)} + b^{(2)}$$

- Form another linear model

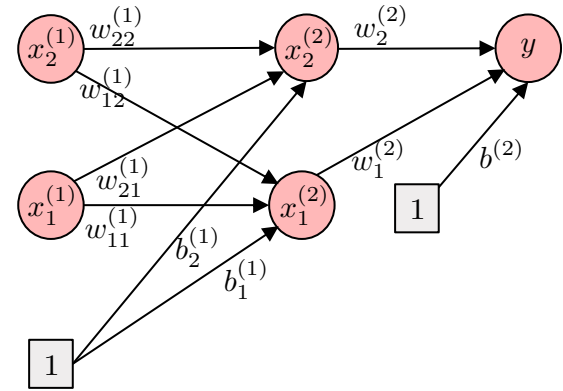
$$y = w_1^{(2)} w_{11}^{(1)} x_1^{(1)} + w_1^{(2)} w_{12}^{(1)} x_2^{(1)} + w_1^{(2)} b_1^{(1)}$$

$$+ w_2^{(2)} w_{21}^{(1)} x_1^{(1)} + w_2^{(2)} w_{22}^{(1)} x_2^{(1)} + w_2^{(2)} b_2^{(1)} + b^{(2)}$$

$$= (w_1^{(2)} w_{11}^{(1)} + w_2^{(2)} w_{21}^{(1)}) x_1^{(1)} + (w_1^{(2)} w_{12}^{(1)} + w_2^{(2)} w_{22}^{(1)}) x_2^{(1)} + w_1^{(2)} b_1^{(1)} + w_2^{(2)} b_2^{(1)} + b^{(2)}$$

- Or $y = \mathbf{w}^{(2)} (\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}) + b^{(2)}$

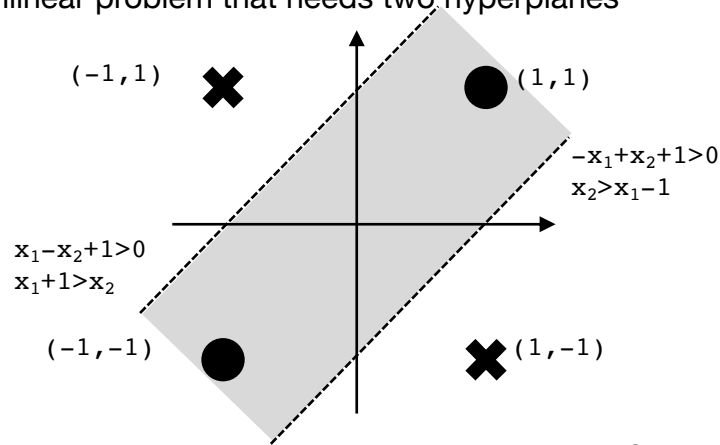
$$= \mathbf{w}^{(2)} \mathbf{W}^{(1)} \mathbf{x} + \mathbf{w}^{(2)} \mathbf{b}^{(1)} + b^{(2)}$$



The First Layer

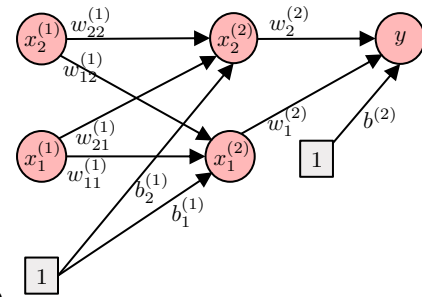
- Nonlinearity in neural networks

- What can we do then?
- The XOR problem
 - A nonlinear problem that needs two hyperplanes

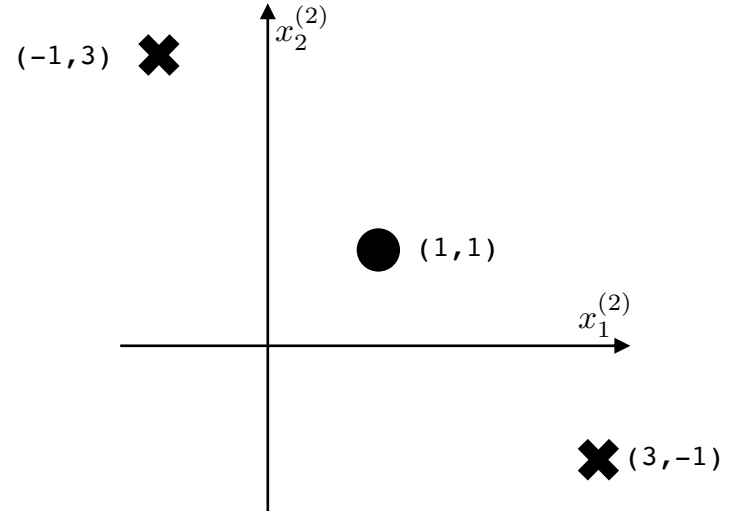


- How do you transform the data into features?
 - It must have something to do with hyperplanes

$$W^{(1)} = \begin{bmatrix} +1 & -1 & 1 \\ -1 & +1 & 1 \end{bmatrix}$$



- The new feature space



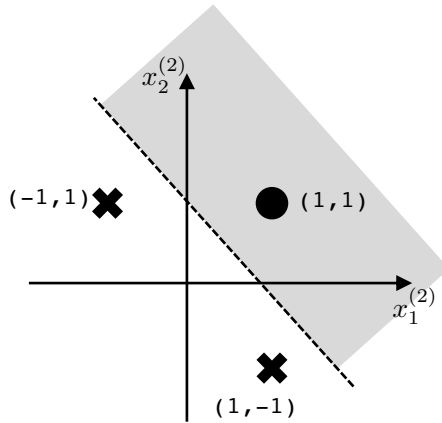
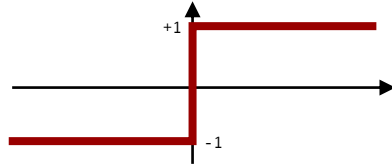
- What happens in the feature space?
 - If it were not for the help from the nonlinearity
 - Still NOT linearly solvable!

The First Layer

- Nonlinearity in neural networks

- With a sign function

$$x_i^{(2)} = \text{sign}(\mathbf{W}_{i,:}^{(1)} \mathbf{x} + b_i^{(1)})$$

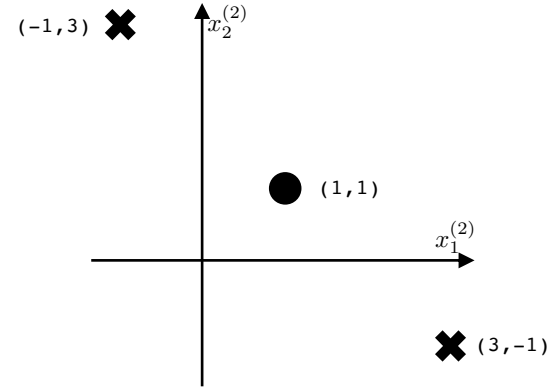
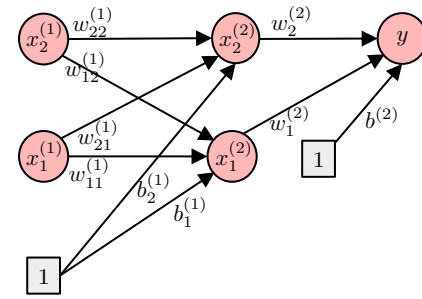
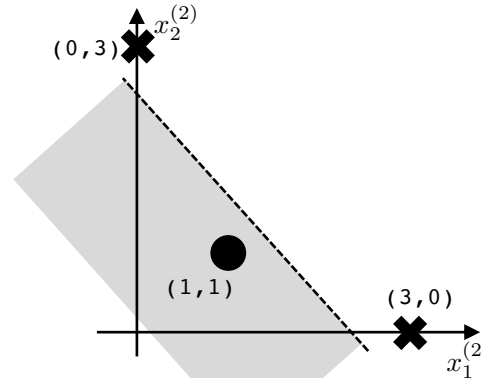
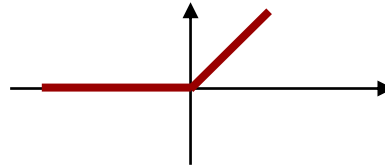


Now they are linearly solvable!

New hyperplane corresponds to the last layer weights

- With a Rectified Linear Unit (ReLU)

$$x_i^{(2)} = \max(\mathbf{W}_{i,:}^{(1)} \mathbf{x} + b_i^{(1)}, 0)$$



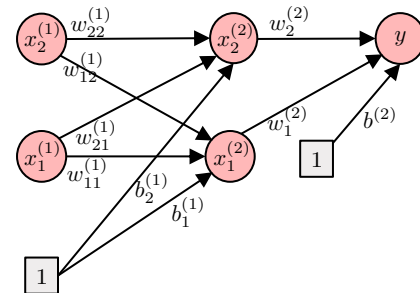
The First Layer

- Nonlinearity in autoencoders

- You can replace the last layer binary output with the input vector
 - Classifier turns into an AE
- Suppose you can estimate these weights of the XOR network by using BP
- Can you also estimate these weights with the AE setup?
- You just saw that nonlinearity can help produce better features
- AE might benefit from it to

$$\arg \min_{\mathbf{W}, \mathbf{W}^\dagger, \mathbf{H}} \mathcal{D}(\mathbf{X} || \mathbf{W} \sigma(\mathbf{W}^\dagger \mathbf{X})) + \lambda_{\mathbf{W}} f(\mathbf{W}) + \lambda_{\mathbf{W}^\dagger} f(\mathbf{W}^\dagger) + \lambda_{\mathbf{H}} g(\mathbf{H})$$

- But it's not clear if the encoder weights are going to be actually create useful features for the following classification
- We will cover this part in the next time



The First Layer

- The pipeline for supervised learning

- Feature engineering
 - Tries to come up with a set of features that best describe the data and the problem
 - There can be many different ways (as you've seen)
 - You never know the quality until you actually test out those features for your problem
- Supervised learning (classification/regression)
 - Tries to come up with the best model to best predict the output variable
 - Even if you use a very nice set of features, nonlinearity can be still involved (e.g. kernel methods for SVM)
 - If you don't like the performance, you never know what to blame
 - It could be because of either bad feature engineering or classifier
- A holistic approach
 - In a multilayer perceptron, you have the first layer dedicated to the feature extraction
 - And the last layer as your supervised learning part
 - What if you just optimize the weights of both layers all together?
 - Then, you can skip the feature engineering part
 - Does this work? When does this approach not work? Why?





Thank You!

